Technische Hochschule Nürnberg

Fakultät Informatik

RaspberryPi CyberSec Lab: Development of a Penetration Testing Platform

Bachelorarbeit im Studiengang Elektro- und Informationstechnik

vorgelegt von

Jonas Schmitt

Matrikelnummer 3619551

Erstgutachter: Prof. Dr. Aykan D. Inan

Zweitgutachter: Jörg Arndt

© 2025

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Technische Hochschule Nürnberg

Hinweis: Diese Erklärung ist mit Originalunterschrift (nicht gescannt) in das Papierexemplar der Abschlussarbeit fest einzubinden. Eine Spiralbindung ist nicht zulässig. Das digitale Exemplar enthält einen Scan der Erklärung mit Unterschrift.

Prüfungsrechtliche Erklärung der/des Studierenden

Angaben des bzw. der Studierenden:

Name: Schmitt Vorname: Jonas Matrikel-Nr.: 3619551

Fakultät: EFI Studiengang: BEI

Semester: 7

Titel der Abschlussarbeit:

RaspberryPi CyberSec Lab: Development of a Penetration Testing Platform

Ich versichere, dass ich die Arbeit selbständig verfasst, nicht anderweitig für Prüfungszwecke vorgelegt, alle benutzten Quellen und Hilfsmittel angegeben sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe. Außerdem versichere ich, dass der Hauptteil des Papierexemplares und des digitalen Exemplares identisch sind. Das digitale Exemplar enthält, falls gefordert, lediglich weitere Anlagen.

Nürnberg,

Ort, Datum, Unterschrift Studierende/Studierender

Erklärung der/des Studierenden zur Veröffentlichung der vorstehend bezeichneten Abschlussarbeit

Die Entscheidung über die vollständige oder auszugsweise Veröffentlichung der Abschlussarbeit liegt grundsätzlich erst einmal allein in der Zuständigkeit der/des studentischen Verfasserin/Verfassers. Nach dem Urheberrechtsgesetz (UrhG) erwirbt die Verfasserin/der Verfasser einer Abschlussarbeit mit Anfertigung ihrer/seiner Arbeit das alleinige Urheberrecht und grundsätzlich auch die hieraus resultierenden Nutzungsrechte wie z.B. Erstveröffentlichung (§ 12 UrhG), Verbreitung (§ 17 UrhG), Vervielfältigung (§ 16 UrhG), Online-Nutzung usw., also alle Rechte, die die nicht-kommerzielle oder kommerzielle Verwertung betreffen.

Die Hochschule und deren Beschäftigte werden Abschlussarbeiten oder Teile davon nicht ohne Zustimmung der/des studentischen Verfasserin/Verfassers veröffentlichen, insbesondere nicht öffentlich zugänglich in die Bibliothek der Hochschule einstellen.

Hiermit ☑ genehmige ich, wenn und soweit keine entgegenstehenden Vereinbarungen mit Dritten getroffen worden sind,

☐ genehmige ich **nicht,**

dass die oben genannte Abschlussarbeit durch die Technische Hochschule Nürnberg Georg Simon Ohm, ggf. nach Ablauf einer mittels eines auf der Abschlussarbeit aufgebrachten Sperrvermerks kenntlich gemachten Sperrfrist

von 0 Jahren (0 - 5 Jahren ab Datum der Abgabe der Arbeit),

der Öffentlichkeit zugänglich gemacht wird. Im Falle der Genehmigung erfolgt diese unwiderruflich; hierzu wird der Abschlussarbeit ein Exemplar im digitalisierten PDF-Format an die Betreuer übermittelt. Bestimmungen der jeweils geltenden Studien- und Prüfungsordnung über Art und Umfang der im Rahmen der Arbeit abzugebenden Exemplare und Materialien werden hierdurch nicht berührt.

Nürnberg,

Ort, Datum, Unterschrift Studierende/Studierender

Datenschutz: Die Antragstellung ist regelmäßig mit der Speicherung und Verarbeitung der von Ihnen mitgeteilten Daten durch die Technische Hochschule Nürnberg Georg Simon Ohm verbunden. Weitere Informationen zum Umgang der Technischen Hochschule Nürnberg mit Ihren personenbezogenen Daten sind unter nachfolgendem Link abrufbar: https://www.th-nuernberg.de/datenschutz/

Abstract

The goal of this project is to develop a flexible and expandible architecture, which can be used for demonstration and practice purposes in all kinds of IT applications. In terms of features, the project focuses on the general user interface and device concept as well as testing environments for WiFi networks. Therefore this thesis includes the documentation of the base program and hardware. Also the basics of WiFi networks and their security, followed by testing of the system with some common cyber attacks. Finishing, the worklog will lay out the development process and the problems encountered along development.

Kurzdarstellung

Diese Arbeit befasst sich mit der Entwicklung einer Testplatform für Cyberangriffe mit dem Fokus auf Funkverbindungen. Im Rahmen den Projekts wurde auf Basis von RaspberryPi OS Software entwickelt, um Umgebungen zu schaffen, in denen verschiedene Cyberangriffe getestet werden können.

Contents

1. Introduction **1**
1.1. Goal 1
1.2. Overview 2

2. Basics of Cybersecurity **3**
2.1. IT Security 3
2.2. Cybersecurity 3
2.2.1. Goals 3
2.2.2. Methods 4
2.3. Threads and Risks in the Cyberspace 4

3. Basics of WiFi **7**
3.1. History 7
3.2. MAC layer 8
3.2.1. Architecture 8
3.2.2. MAC Frames 8
3.2.3. Medium Access 10
3.2.4. Connection Establishment and Termination 10
3.3. PHY layer 11
3.4. Security 12
3.4.1. Encryption 12
3.4.2. Authentication 13

4. Documentation **15**
4.1. Hardware 15
4.2. Software 16
4.2.1. Overview 16
4.2.2. UI Program 17
4.2.3. Encoder Program 18
4.2.4. Shell Scripts 19
4.2.5. Bluetooth 21
4.2.6. Juice Shop 21
4.2.7. MQTT Communication 21
4.2.8. ESP32 22

4.2.9. Miscellaneous . . . 22

5. Development . . . **23**
5.1. Project Inception . . . 23
5.2. User Interface . . . 24
5.3. Network Management . . . 25
5.4. Rotary Encoder . . . 26
5.5. ESP32 . . . 27

6. Pentesting . . . **29**
6.1. Attacks . . . 29
6.1.1. War Driving . . . 29
6.1.2. Rogue Access Point and Evil Twin . . . 29
6.1.3. Cracking WEP . . . 30
6.1.4. Cracking WPA/WPA2 . . . 31
6.1.5. Man-in-the-Middle . . . 31
6.2. Methods . . . 31
6.2.1. Cracking WEP . . . 31
6.2.2. Cracking WPA/WPA2 . . . 34
6.3. Results . . . 35
6.3.1. Cracking WEP . . . 35
6.3.2. Cracking WPA/WPA2 . . . 36

7. Discussion . . . **37**
7.1. Importance of Cybersecurity Education . . . 37
7.2. Pentesting Results Evaluation . . . 38
7.3. Usecase . . . 38

8. Summary . . . **39**

9. Outlook . . . **41**
9.1. UI Program . . . 41
9.2. Input Processing . . . 41
9.3. Features . . . 42

A. Supplemental Information . . . **43**
A.1. User Manual . . . 43
A.2. WiFi Basics . . . 47
A.3. Documentation . . . 47

List of Figures . . . **49**

List of Tables . . . **51**

List of Listings 53

Bibliography 55

Glossary 57

Chapter 1.

Introduction

The modern world is moving towards digitalization at a high pace, which aims to make it smart and interconnected. This tranformation comes with the ever increasing use of connected devices and services, which oftentimes carry sensitive data and are not always protected sufficiently. As a result, there is a rising number of cyber crime incidents and subsequent damages estimated to more than $1 trillion worldwide.

Compoundig on top of the situation is a mismatch between demand and supply of Cybersecurity professionals and a general public, which is in large parts not aware of the risks posed by the use of connected IT technology. Cybercriminals can often easily exploit these unaware people to gain access to resticted systems and data. To combat this problem, there's a need for training Cybersecurity professionals, as well as to raise awareness in every level of education and the general public. As a method for raising awareness, teaching priciples of Cybersecurity and the methods to apply them, live demonstration and hands-on learning with realistic risk scenarios can be an effective tool. [Mari 24][Cybe 23]

1.1. Goal

An effective method to ensure the security of a system is pentesting[1], which helps to discover weaknesses in soft- and hardware.

The goal of this project is to develop a platform with a flexible and expandible architecture, which can be used for demonstration and practice of pentesting in Cybersecurity education. A Raspberry PI will be used as the main device to create a platform, which is portable and easy to use. It should be able to reliably set up testing environments for various fields of application, to allow a lecturer to demonstrate cyberattacks and students to practice penetration testing in a realistic enviroment. In recent years, trends like the Internet of Things (IoT) have produced many headless[2] devices, which are often linked to smartphones for user interaction, using WiFi or Bluetooth for communication. For this reason, wireless

[1] penetration testing, see chapter 2
[2] without any means for direct interaction

communication security in the form of WiFi technology will be implemented as the first application specific testing environment. To teach secure design of web applications, the device should be able to host a server with the OWASP Juice Shop, which again ties into the security of IoT devices, because they are oftentimes accessed via web interfaces, which are often the source of vulnerabilities[Hell 23, page 174].

1.2. Overview

- chapter 2 explains the principles of it security and the difference between Cybersecurity and it security.
- chapter 3 explores the history of wifi technology and explains its architecture, function and security mechanisms.
- chapter 4 explains the architecture of the hard- and software of the RCSL.
- chapter 5 documents the development process and the challenges faced along the process.
- chapter 6 explains the penetration tests conducted and documents the results of testing them on the RCSL.
- chapter 7 outlines the challenges of Cybersecurity education and discusses usecases for the RCSL in the field.

Chapter 2.

Basics of Cybersecurity

This chapter briefly explains the terms of IT- and Cybersecurity and gives an overview of their goals and methods. A summary of common threads in the Cybserspace is given as well.

2.1. IT Security

IT Security describes the protection of digital data and computer- and communication systems from unauthorized access. Different principles are adhered to and methods and techniques applied in order to prevent the theft, interception, manipulation and loss of data and systems. [Pogu 13]

2.2. Cybersecurity

IT- and Cybersecurity are similar in that both have the same goals and apply the same principles and methods. While IT Security focuses on classical computer systems and networks, Cybersecurity focuses on data and systems in the Cyberspace. Included in the Cyberspace are all systems and data connected to the internet.

2.2.1. Goals

The goals of Cybersecurity include but are not limited to the well known CIA triad, which is an acronym for Confidentiality, Integrity, and Availability [Oriy 17]. Confidentiality describes data only beeing accessible by the persons, who are meant to do so. Integrity describes data beeing unchanged after storage or transmission, meaning the data was not corrupted or manipulated. Availability describes data and systems beeing accessible with the expected performance, whenever required.

2.2.2. Methods

Some methods for achieving the goals of Cybersecurity are:

- Security Awareness is the knowledge of the threads present in the Cyberspace and methods for protection. "Awareness of the risks and available safeguards is the first line of defence for the security of information systems and networks" [Pogu 13, page 26].
- Encryption is a cryptographic[1] method, which takes the readable data, called plaintext, and makes it unreadable for anyone besides the legitimate recipient. The encrypted text is called ciphertext and can be made readable again, by decrypting it. To en- and decrypt a message, the sender and recipient have knowledge of a secret, often a passphrase, which controls en-/decryption and is called key. [Watj 18, page 1]
- Authentication is the act of a user "performing some sort of action which proves that the claim a user is making about who they are is actually valid" [Oriy 17]. This is oftentimes done through the use of a password, in the case of WiFi networks, the knowledge of the password proves, that a user is allowed to access the network.
- Monitoring with anomaly detection can be used to detect the unauthorized access of systems and networks. In the network domain so called Intrusion Detection Systems (IDS) employ predictive models to recognize malicious activity inside a network [Geek 23b].
- Incident Response is performed in case of a system breach in order to limit damage and secure the system.

2.3. Threads and Risks in the Cyberspace

There are various types of Cybercriminals with different motives and intentions. Motives of Cybercriminals can vary but commonly are financial, political, social or military. With these motives the intentions of an attack often are:

- blackmail
- espionage
- theft
- fraud
- sabotage
- corruption and insider deals

The successrate and risk of an attack are in large parts determined by a targets attack area, which is the combination of the present vulnerabilities and attack vectors. The term

[1] cryptography = science of secretive writing

vulnerability describes a weakspot, which can be exploited by the attacker to break or circumvent security mechanisms. An attack vector is the approach, an attacker takes for carrying out an attack. With regards to Network Security some common attack vectors are:

- Denial of Service (DoS) attacks availability by preventing or limiting the functional capability of a system.
- Cracking describes the act of breaking or circumventing security mechanisms, e.g. reverse engineering passwords.
- Brute-Force is a technique for reverse engineering passwords by trail-and-error.
- Spoofing refers to fakeing identities, achieved by the obfuscation of the own identity or the replication of another identity.
- Sniffing is the action of collecting all openly available data, e.g. recording unecrypted wifi frames.
- Man-in-the-Middle (MitM) attacks involve injection of a malicious device into a line of communication, which subsequently can manipulate and eavesdrop on the traffic.
- Social Engineering is a form of manipulation which abuses psychological mechanisms like trust, curiosity or shock to exploit victims.

[Pogu 13, page 16-20]

Chapter 3.

Basics of WiFi

This chapter explains the basics of WiFi technology. Insight into the history and development of WiFi is given and its functionality and security mechanisms are briefly explained.

3.1. History

In 1985 the american Federal Communications Commission ruled serveral bands in the 2.4 GHz radio band to be unlicensed, meaning everyone can broadcast radiosignals in these frequencies. At the beginning of the 90's, this lead to the development of different wireless communication protocols by multiple companies. To ensure interoperability between these protocols, the IEEE 802.11 standard was created and released, and WLAN was born. In 1999 the Wi-Fi Alliance (WFA) was founded to further improve compatibility, nowadays consisting of over 750 international companies. [Sank 21, page 1-3], [Wiki 25]

Since its inception over 25 years ago, Wi-Fi has gone through constant development, leading to six major generations of the technology. Improvments in speed, efficiency and security have lead to the technology becoming a "fundamental utility [...], that everyone [...] expects to be available everywhere" [Sank 21, page 1]. WiFi connects billions of devices, like computers, smartphones, game consoles or IoT devices to the internet, generating a value of around 4.9 trillion USD worldwide [Sank 21, page 1].

WiFi operates at the physical (PHY) and the medium access control (MAC) layer of the OSI model, sending ethernet frames over radio waves in the 2.4 and 5 Ghz frequency band. The MAC layer is responsible for seting rules on tranmission and reception of data, connection management and power management. Data sent on the MAC layer then gets converted to radio waveforms by the PHY layer, which is also responsible for converting received data back to the MAC protocol. [Sank 21, page 4-7]

3.2. MAC layer

Devices in the MAC layer are adressed with a MAC address, which consists of 12 hexdecimal characters and is unique for every network interface controller (NIC).

A MAC adress could look like this: `61:A3:E2:73:9A:F3`.

3.2.1. Architecture

Commonly WiFi networks have access points (AP), which are connected to the internet via ethernet, and stations (STA), which are provided with internet access over WiFi by the AP. WiFi has a MAC layer, which is designed to be flexible and can be configured in the following topologies:

- **BSS**: Basic Service Set
- **IBSS**: ad hoc / peer-to-peer / Independent BSS
- **WDS**: Wireless Distribution System
- **MBSS**: Mesh BSS

BSS describes the most common and basic configuration, where multiple STAs are connected to one AP, which has wired connection to the internet. IBSS networks dont have any APs, instead the STAs are connected directly to each other. WDS can be used, when the reach of an AP needs to be extended, but there is no wired connection available. A secondary AP can be installed, which receives backhaul connection from the first AP. Mesh Networks are complex BSS networks with multiple APs, which the STAs can hop between. This type of network can improve the signal coverage and resilience and is often used in companies or public institutions. [Sank 21, page 8-9]

3.2.2. MAC Frames

When data is sent over the MAC layer, it gets divided into chunks, which are called MAC Frames. MAC Frames have three components: the header, the body and the frame check sequence. The header contains information about the source and destination, the sequence and serveral control functions. In a BSS network it has the following functions:

- **Frame Control** contains 16 Bits which serve different control functions

  protocol version describes which version of the MAC protocol this frame uses

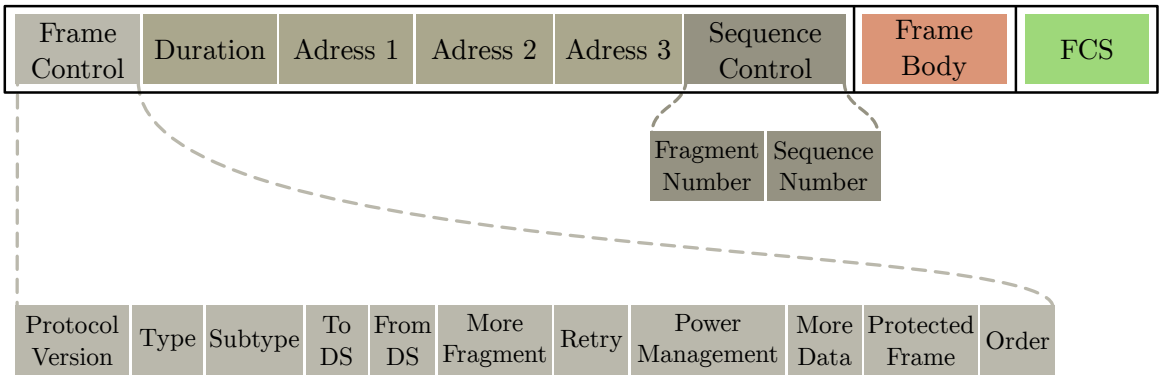


Figure 3.1.: MAC Frame of BSS WiFi Network

type describes the type of MAC frame is beeing used: there are data, management, control and acknowledgment frames

subtype describes the subtype of the frame, e.g. association request or beacon frame

from/to ds indicates the direction of travel

more fragment indicates more following fragments

retry indicates that the current frame is a retransmission

power management gives information about the power state of the STA

more data indicates more frames are to be transmitted

protected frame indicates whether a frame is encrypted

order set to 1 if the order of processing is important

- **duration** indicates the time of the transmission, therefore how long the medium is occupied
- **adress 1** contains the receiver address (RA), which can be the MAC address of the destination STA or the BSSID of the AP, depending on the direction, the frame travels
- **adress 2** contains the transmitter adress (TA), which can be the BSSID or the source address.
- **adress 3** contains either the source or destination address
- **sequence control**

  fragment number ensures the correct processing sequence of the fragments

  sequence number number of the frame

The Frame body then contains the payload, meaning the actual information to be sent. Every type of MAC frame is configured a different way to suit their specific function. "While data frames are meant to carry data, management frames are used for connection management and control frames [...] for triggering specific actions. Control frames have a smaller MAC header and no payload data [...]. Management frames have a structured payload composed of some fixed length fields and one or more variable length fields called information elements (IEs)" [Sank 21, page 12]

After the frame body comes the frame control sequence (FCS), which ensures the integrity of the frame. The FCS is a cyclic redundancy check (CRC), which is number calculated from the frame header and body content. When the frame is compromised in traffic, the result of the CRC changes, therefore the faulty frame can be identified. [Geek 23a] [Sank 21, page 10-12]

3.2.3. Medium Access

In LAN and WLAN networks all devices share the same medium, therefore only one device at a time can transmit information. Ethernet networks use full-duplex transceivers, meaning a device can transmit and receive data at the same time. This allows the transceiver to transmit and simlutaniously listen for collisions with other frames. Collision detection is quite effective, but can not be used in WiFi networks, because WiFi transceivers are only capable of half-duplex transmission. WiFi instead uses a medium access control referred to as carrier sense multiple access collision avoidance (CSMA-CA).

When using this technique, the medium gets blocked during transmission and is freed by an acknowledgment frame, sent by the receiving device. All STAs start off in the random backoff state, where they listen for traffic. After the random backoff counter has expired and no traffic is present, the STA will start transmission. CSMA-CA can be described by the way "humans converse in a group, where every individual would first listen to check if someone else is talking, wait for the person talking to finish, and then start talking after a brief random pause" [Sank 21, page 12]. [Oriy 17] [Sank 21, page 12-17]

For addressing a STA, its MAC address needs to be known. For sharing the MAC address of a STA with all other STAs on the network the Address Resolution Protocol (ARP) is employed, which broadcasts the STAs MAC addresses, allowing all other devices to save it.

3.2.4. Connection Establishment and Termination

Before a connection can be established, the STA first needs to discover the AP it wants to connect to. This can be done by either active or passive scanning of all available channels.

During active scanning, the STA transmitts a management frames, called probe requests on all channels to all SSIDs in the vicinity. The APs answer with a probe respose frame, which contains the AP's SSID, BSSID and its capabilities.

Passive scanning works by the STA listening for beacon frames. Beacon frames are periodically sent out by APs and also contain information on SSID, BSSID and capability. [Sank 21, page 18]

Once the desired network is discovered the STA decides which AP to connect to, based on parameters, like the receive signal strength indicator (RSSI) or the APs capacity, performance and security. The STA then sends an authentication request, which the AP gives an authentication response to. If the credentials of the STA are valid and the authentication is successful, the STA proceeds to send an association request. The association response, sent by the AP, then indicates, if the connection was established successfully.

To terminate a connection, either the STA or the AP can send a disassociation or deauthentication frame, "containing a reason code that specifies the reason for termination" [Sank 21, page 20]. [Sank 21, page 19]

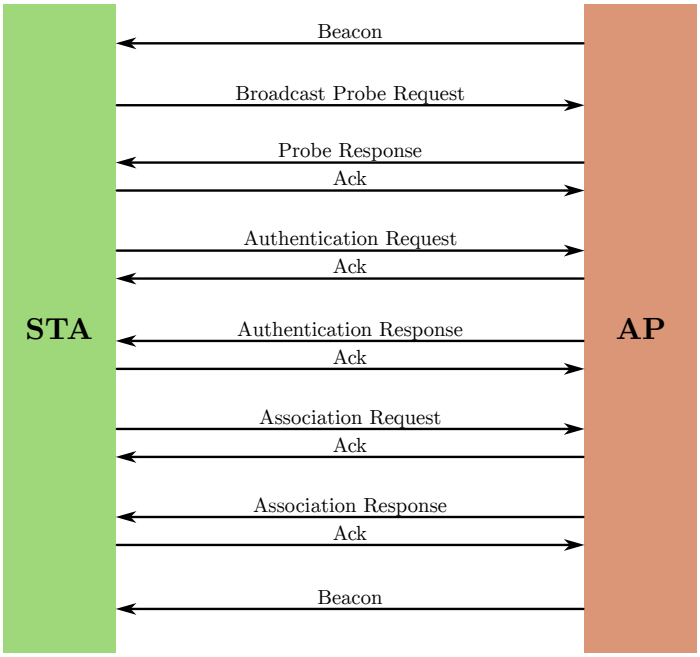


Figure 3.2.: Connection Establishment

3.3. PHY layer

The WiFi PHY layer is based on radio frequency (RF), therefore WiFi transceivers contain a transmitter and receiver circuit, as well as an antenna. When sending data, MAC frames

are encoded and modulated onto a RF waveform which is then transmitted via the antenna. On arrival of data, the receiver circuit decodes the data, extracts the MAC frame, and provides it to the MAC layer. The first generations of WiFi used to transmit in the 2.4 Ghz band, but due to it beeing used by many other devices, such as cordless phones, computer peripherals, bluetooth devices or microwave ovens, WiFi 4 started using the 5 Ghz band, in order to improve connectivity. [Sank 21, page 5,34,35] With every generation of WiFi, the PHY layer components are modified to improve performance and efficiency, but due to the PHY layer not beeing majorly significant for cybersecurity, it will not be described further in this thesis.

3.4. Security

Through the widespread adoption of WiFi technology, for example in smart homes, infrastructure, public spaces, industry and goverments, the security of WiFi networks plays an ever increasing role. Therefore principles of cybersecurity need to be applied, in order to guarantee that networks are safe, reliable and performant. Because of its wireless nature, "[i]t is difficult to confine the data-carrying radio waves to be within the physical security perimeter of [an] organizsation" [Sank 21, page 103], so anyone in the vicinity of a WiFi network could just intercept and manipulate network traffic. To ensure integrity and confidentiality, WiFi two main security mechanisms, which are encryption and authentication. [Sank 21, page 103]

Note: Since this thesis is focused on private networks, the following summary of WiFi security mechanisms will not include enterprise authentication protocols.

3.4.1. Encryption

The first "secure" WiFi standard, called Wired Equivalent Privacy (WEP) uses a shared key, with a RC4 stream cipher algorithm for message encryption and a 32-Bit CRC as a integrity check (IC). Due to serveral vulnerabilities of WEP, including the RC4 algorithm, a lack of authentication and no key management, the second standard, wireless protected access (WPA), was developed.

WPA still uses the RC4 cipher for compatibility reasons, but the encryption mechnism was updated, now called Temporal Key Integrity Protocol (TKIP), it uses a pairwise master key, which is generated for each STA after it has authenticated successfully. TKIP also dynamically changes the encryption key for each packet, which in combination with a longer 128-Bit preshared key (PSK) fixed some of the weaknesses of WEP. [Oriy 17]

WPA2 could be considered as the first proper WiFi security standard, since it replaced the weak RC4 algorithm with AES-CCMP[1] encryption. AES-CCMP encrypts data blockwise and uses a compbination of substitution and transposition algorithms to achieve high security. WPA2 security also added protected management frames, so not only data frames, but also management frames are encrypted. This helps to secure networks against DoS attacks with deassociation or deauthentication frames.

The newest WiFi security standard is WPA3, which aims to fix vulnerabilities of WPA2 and future proof the technology. Most changes for WPA3 have been made in authentication mechanisms, WPA3 added the optional support of AES-GCMP[2] encryption, which is more performant and even safer than AES-CCMP. [Sank 21, page 103-117]

3.4.2. Authentication

WEP authentication is only one-sided, where STA has to encrypt a package with the correct key, in order to complete association with the AP. The same key is used for encryption, making the whole system vulnerable, when ether the authentication or the encryption is exploited.

WPA and WPA2 networks use an authentication mechanism, that is two sided, where STA and AP exchange identity and capability. After entering the correct passphrase a password-based key derivation function then generates the PSK for encryption. This works by deriving the PSK "as a function of the Wi-Fi password, SSID, and SSID length using a password-based key derivation function" [Sank 21, page 106].

Because the PSK is transmitted during the handshake on connection establishment, PSK is potentially vulnerable to offline and dictionary attacks, when a weak passphrase is used. To future proof the technology, WPA3 replaces PSK with Simultaneous Authentication of Equals (SAE). SAE works by choosing an element from a finite cyclic group (e.g. point on an elliptic curve) based on the password and performing a Diffie Hellman exchange to generate a pairwise master key. This mechanism allows authentication without the transmission of any information related to the password and is performed for every session to ensure forward security. [Oriy 17], [Sank 21, page 115-116], [Hark 08]

Note: A full summary of the different security specifications of each WiFi generation, can be found in table A.1.

[1] Advanced Encryption Security - Counter mode CBC MAC Protocol
[2] AES - Galois Counter Mode Protocol

Chapter 4.

Documentation

This chapter documents the hard- and software of the RaspberryPI Cyber Security Lab (RCSL), wherein the hardware section describes the hardware components, which the device is built from and the software section explains the architecture of the software and its functionality.

4.1. Hardware

The figure below shows the hardware components, used in the project, are a RaspberryPI (RPI), a USB network card, an ESP32 microcontroller, a display and a rotary encoder.

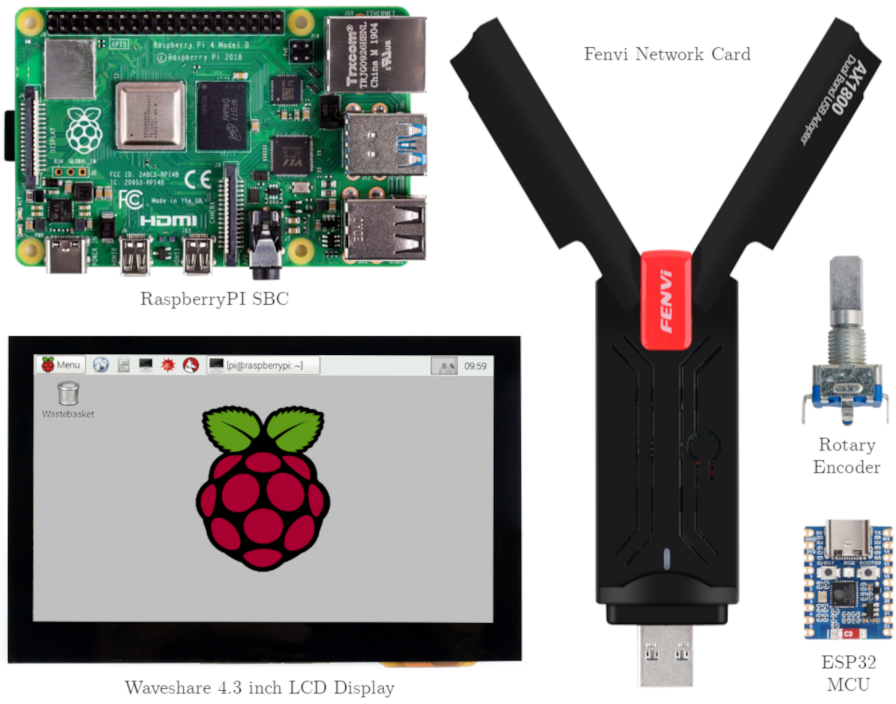


Figure 4.1.: Hardware Components

As the main computer the RaspberryPI 4B with 8GB of RAM was chosen due to its ease of use, linux based operating system and widely available documentation.

The RPI has an integrated WiFi and Bluetooth card, as well as onboard ethernet networking. However to support all WiFi standards an external network card is needed, since the internal card of the RPI does not support the ciphers used by WPA3. The Fenvi AX1800 USB WiFi card was chosen for its compatibility with the linux kernel, according to [NR 25], and low cost.

For some types of attack, such as cracking WPA/WPA2[1], a WiFi station, which is connected to the test network is needed, which in this case is an ESP32 microcontroller unit (MCU). Selecting from the large offer of ESP32 MCUs, again size was taken into account and the ESP32-C6 Mini by Waveshare was chosen for its small size and good feature set. It connects to the RPI's GPIO pins using simple jumper cables, which supply the ESP32 with power and establish an UART connection.

Another device connected to the RPI's GPIO header is the rotary encoder. On its five pins, it receives 3.3V supply voltage and outputs a gray code, indicating the direction of rotation, on two of its outputs. The fifth pin is used as the input for the pushbutton action of the encoder.

The hardware components are mounted in a 3d printed case, like shown in **??**

4.2. Software

Note: Due to my limited experience in software development, the project does not focus on performance or software quality. The software is written to implement the wanted functionality and should be seen as experimental, since it is neither optimised for performance, nor have proper error handling mechanisms been implemented.

4.2.1. Overview

The software of the project is mostly modular and it consists of a central program, which runs the user interface (UI) and then executes various shell scripts, with which the functionality is implemented. As to be seen in fig. 4.2, besides the UI program, there are programs for MQTT communication and input processing, also the services of the Network Manager and Juice Shop project are used.

Note: only the UI program is written object oriented and only Menu and MenuOption are object classes.

[1] see chapter 6

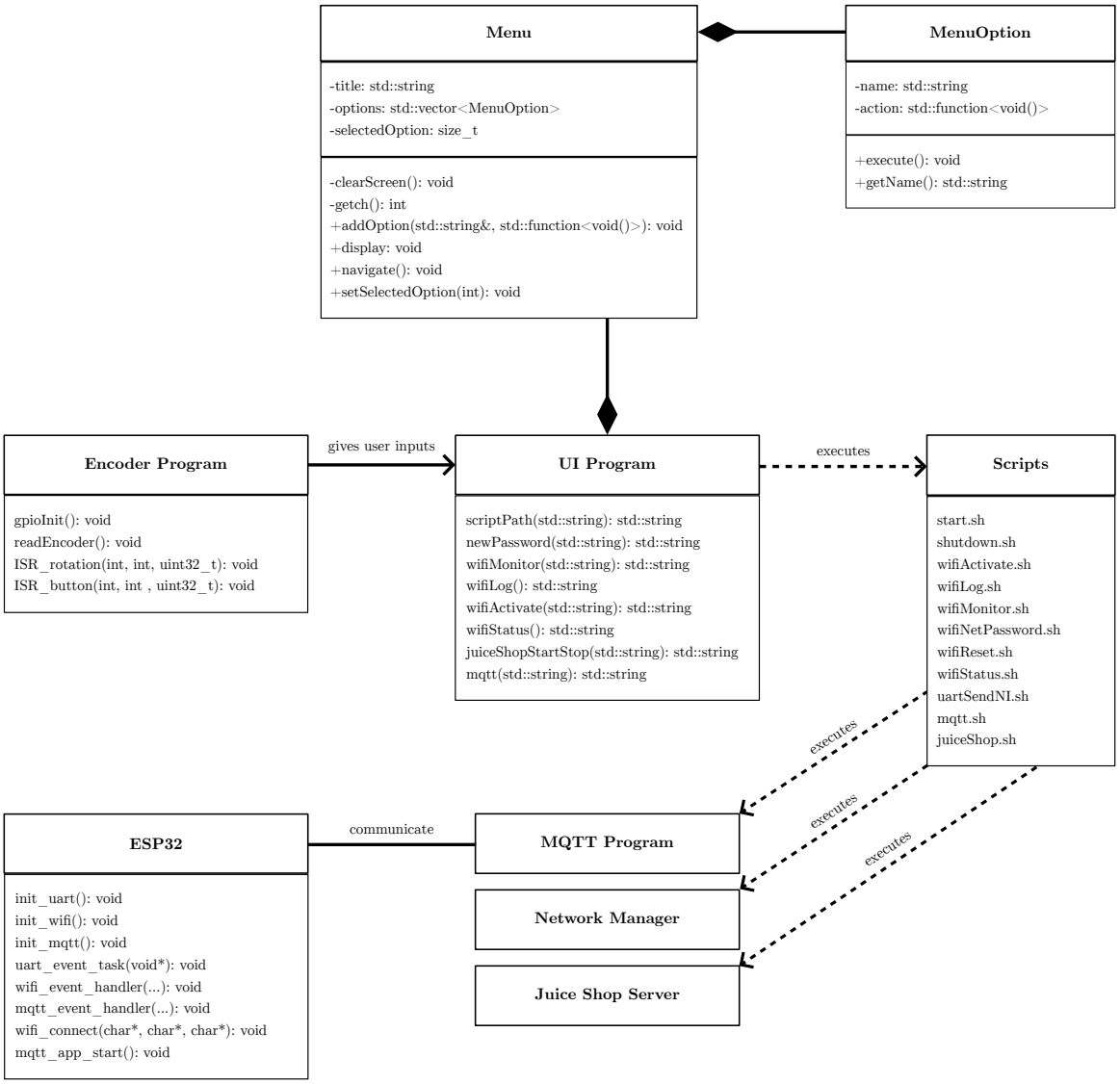


Figure 4.2.: Software Architecture

4.2.2. UI Program

Upon booting the device, a systemd service executes start.sh, which starts the encoder program and the UI program with the output of the encoder program piped into the input of the UI. The UI displays the main menu, depicted in fig. 4.3, which the user can interact with by using the rotary encoder. Each page of the UI is a "Menu" object, each menu has a title, options stored inside a vector of the type "MenuOption", a variable to track the currently selected option, a method to add options to a menu and methods for navigation, input capture and displaying. MenuOption objects have a name, as well as an action of the std::function type, which can be executed by calling the execute() method. In the main of the program, the constructors for menues and the addOption() methods are beeing used, to

create the different menues and their submenues. An overview of the menues and options, currently present inside the UI program can be seen in table A.2. The program starts by calling the navigate() function of the main menu, which then displays all available submenues and menu options. All submenues work in the same way as the main menu, getting their respective navigate() called upon selection. During navigation the UI peridiodically prints the available options, using unicode to highlight the currently selected option as well as to give contrast to the title of the menu.

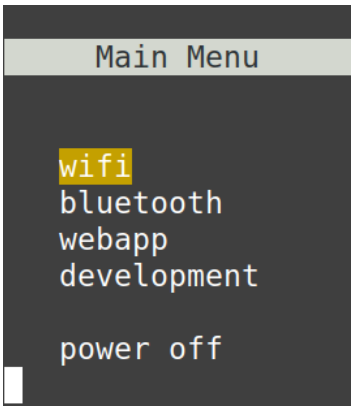


Figure 4.3.: Main Menu

More examples for the UI can be found in the appendix, for example fig. A.1.

When an action, like activation of a network, is executed, main calls the according shell script with the system() function (see 4.2). Scripts, which require an input, are called with a specific function, that pipes the according information into the script (see 4.1).

Listing 4.1: wifiActivate function in the UI program

```
std::string wifiActivate(std::string connection){
    std::string command = "echo " + connection + " | " + scriptPath("wifiActivate");
    return command;
}
```

Listing 4.2: use of wifiActivate for WEP network

```
      wifiActivateMenu.addOption("WEP", [](){system(wifiActivate("WEP").c_str());});
```

4.2.3. Encoder Program

For input the RCSL uses a rotary encoder with a pushbutton, turning the encoder moves the menu selection up or down, the pushbutton executes the selected option (see fig. 4.4). The encoder is connected to the RaspberryPIs GPIO interface, its rotation is segmented with clicks, similar to the scroll wheel of a computer mouse. Like shown in fig. 5.2, rotation between two clicks outputs one complete grey code sequence on the signals DT and CLK,

which is processed by the encoder program. This happens within the rotation handling interrupt service routine (ISR), shown in 4.3, which is executed, when DT outputs a high-low transition.

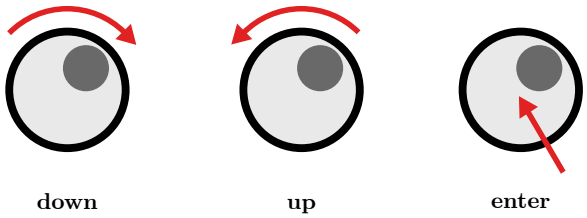


Figure 4.4.: Navigation Scheme

The ISR checks the state of CLK, which determines the direction the encoder was turned: CLK=0 means, the encoder has been turned clockwise, CLK=1 means counter-clockwise (see fig. 5.2). Subsequently a command in form of a string, representing an ANSI Escape Code, is printed. Escape Codes are used as an output for peripheral buttons, which are not a character, for example `"\033[A"` describes the pressing of the downwards button on the keyboard. As previously explained, the output of the encoder program is piped into the input of the UI, therefore rotating the encoder leads to the UI recieving the codes for up and down. The pushbutton action is processed in a similar way, with an ISR outputting a newline symbol to the UI.

Listing 4.3: Rotation handling ISR

```
void ISR_rotation(int gpio, int level, uint32_t tick) {
    if (level == PI_LOW) {
        volatile int clk = gpioRead(CLK);
        if (clk == 0) {
            std::cout << "\033[A" << std::flush;   //Arrow Down
            //std::cout << "down";  //Arrow Down
        } else {
            //std::cout << "up";    //Arrow Up
            std::cout << "\033[B" << std::flush;   //Arrow Up
        }
    }
}
```

Because the commands used for navigation are ordinary ANSI escape codes, the program can be easily tested on any computer by navigating the UI with the keyboard.

4.2.4. Shell Scripts

The wifiActivate.sh script is used to open wifi hotspots, where the user can choose the type of Wifi standard beeing used. It receives an input command and then establishes a WEP, WPA, WPA2 or WPA3 network. On first use, it sets up a network configuration, which just gets activated on repeated use, as seen for WEP networks in the code extract 4.4.

Listing 4.4: extract of wifiActivate.sh

```
#!/bin/bash

scriptdir="$(dirname "$0")"

read -n 4 connection

if [ "$connection" == "WEP" ]; then
   exists=$(nmcli connection show | sed -n '/Wifi-WEP /p')
   if [ -z "$exists" ]; then
      sudo nmcli device wifi hotspot con-name "Wifi-WEP" ssid "WEPnetwork" password "
          TestSetup123" ifname wlan1
      sudo nmcli connection down Wifi-WEP
      sudo nmcli connection modify Wifi-WEP 802-11-wireless-security.key-mgmt none
      sudo nmcli connection modify Wifi-WEP 802-11-wireless-security.group wep104,
          wep40
      sudo nmcli connection modify Wifi-WEP 802-11-wireless-security.auth-alg shared
      sudo nmcli connection modify Wifi-WEP 802-11-wireless-security.wep-key0 "test1"
      sudo nmcli connection modify Wifi-WEP 802-11-wireless-security.pmf 1
   fi
      sudo nmcli connection up Wifi-WEP
```

After a WiFi hotspot has been activated, a command with the connection details is sent to the ESP32 over uart. This functionality is implemented by the uartSendNI.sh script, which gets called by wifiActivate.sh (see line 17 of 4.5). Receiving the network information, the ESP32 will then connect to the provided network and act as a communication partner for the RPI.

With the wifiReset.sh script, the currently active network connection is turned off, which gets called by the deactivate option in the WiFi menu.

To display the current status of the network, meaning its security standard, SSID and password, the wifiStatus.sh script is used. It uses the `nmcli dev wifi show-password` to output to the UI, which then looks like the following:

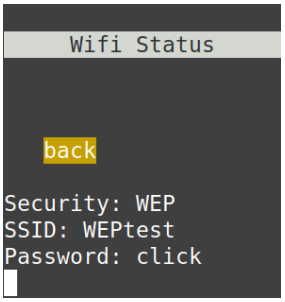


Figure 4.5.: WiFi status menu

When activating monitoring, wifiMonitor.sh uses the `iw event -T` command to log devices connecting and disconnecting from the network. Selecting the "monitor" option, the user is presented with the options of de-/activating the network monitoring as well as viewing or deleting the monitoring log.

Once a network has been cracked, the wifiNewPassword.sh script can be used to give a new random password to the selected network inside the change password menu. The script then pulls a random password from a password list (see 4.5), containing popular passwords, that have been filtered to work with the specification of each network standard. WEP networks use passwords with 5 or 13 characters, while WPA and WPA2 networks need passwords with at least 8 characters and WPA3 networks works with passwords of every length.

Listing 4.5: extract from wifiNewPassword.sh

```
randNumber=$((RANDOM % $(wc -l < $pwddir/passwordsWEP.txt) +1))
newpassword=$(sed -n "${randNumber}p" $pwddir/passwordsWEP.txt)
sudo nmcli connection modify Wifi-WEP 802-11-wireless-security.wep-key0
    $newpassword
```

4.2.5. Bluetooth

Note: the Bluetooth features, originally planned, have not been implemented due to time constraints and may be implemented in the future.

4.2.6. Juice Shop

Inside the webapp menu the JuiceShop and MQTT communication can be found.

Juice Shop is a project by the Open Worldwide Application Security Project (OWASP). It is "a web application with a vast number of intended security vulnerabilities, [which] is supposed to be the opposite of a best practice or template application for web developers: It is an awareness, training, demonstration and exercise tool for security risks in modern web applications" [Kimm 25]. When activating the JuiceShop, the juiceShop.sh script will start up the server for the OWASP Juice Shop project and open a hotspot, which can be used to access the simulated webstore.

Note: futher information about the project can be found in [Kimm 25], which is the official companion to the Juice Shop.

4.2.7. MQTT Communication

When a WiFi network has already been activated from the WiFi menu, a simulated MQTT communication can be started from the MQTT menu. Starting a conversation will again send a command via UART to the ESP32, prompting it to connect to the MQTT broker, running on the RPI. Following up, uart.sh starts the uart program, which begins an infinite MQTT conversation with the ESP32.

4.2.8. ESP32

The code for the ESP32 is based on the code examples, provided by Espressif Systems on the official ESP-IDF github [Syst 23]. It includes event handling for UART, WiFi and MQTT and a client for MQTT. The UART event handler has been modified to process the commands sent by the RPI and passes connection information to the WiFi and MQTT instances.

4.2.9. Miscellaneous

The development menu contains options used during development, these can be adjusted on the fly inside the development.sh script.

When selecting the shutdown option, all running processes are stopped and the shutdown.sh script calls `shutdown now` to turn off the device.

Chapter 5.

Development

5.1. Project Inception

The initial concept for the project revolved around the use of a collection of SD-Cards, containing operating systems tailored for specific usecases. The user would be able insert the wanted SD-Card into the RPI, which then would execute script on startup to configure the operating system for a specific scenario.

With the usability of this system appearing quite cumbersome, it was instead opted to develop a user interface (UI), which would execute the wanted shellscripts during runtime. To implement the UI, it was decided on the use of a simple terminal-based interface, which would be printed directly into the shell. This approach would facilitate straightforward navigation through the use of a dial and pushbutton combination, providing an intuitive means for users to interact with the system. The program should be written in C or C++ due to my familiarity with both languages.

From this concept the following 3D-model was created to visualize the project idea:

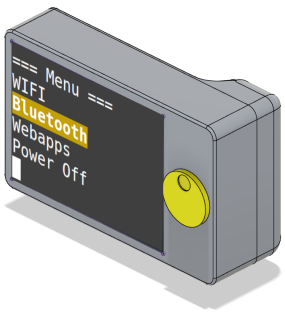


Figure 5.1.: Concept Model

5.2. User Interface

To display the UI, the menues need to be printed into the terminal, for which a simple loop, that peridiodically clears the screen and prints the options of a menu, was decided on. Subsequently a fitting software architecture needed to be designed, which can model the variety of menues and their respective options with individual actions. The menues need to display options, which require the capability of executing various actions, including the transition to other menues and the execution of shellscripts and programs. Since the command design pattern at this point was unknown to me, the modeling of the menues with a uniform class turned out quite challenging. Therefore it was decided to model the menues as classes, containing objects of the MenuOption class, and utilize the std::function for action execution.

In order for the device to be usable with the build in screen and selector dial, the RPI needs to be configured to run and display the UI program on startup. Execution of the program on startup is implemented with the help of a systemd service. For the setup said service an according configuration file needs to be added to */etc/systemd/system*, which is activated by running the **`systemctl enable [service].service`** command.

Instead of a BIOS[1], the RPI employs a configuration file (*/boot/firmware/config.txt*), which is read on startup to initialize the hardware and operating system. To this file the following lines were added:

```
dtoverlay=waveshare35a
framebuffer_width=200
framebuffer_height=120
```

Line one sets up a hardware overlay to initialize the display, lines two and three set the resolution of the display. With the resolution set to the factory specification of 800x480, the UI is displayed quite small, thus poorly suited for the usecase. Since the OS runs in terminal without a graphic server, there is no simple option for scalation of the UI. The scale problem was fixed with a workaround, which involves the resolution inside the *config.txt* to be set to 200x120, therefore forcing the scalation of the UI.

Initially it was planned to use the display in vertical orientation, however, attempts to rotate the terminal by configuration inside *config.txt* or the kernel command line (*/boot/-firmware/cmdline.txt*) were unsuccessful.

[1] Binary Input Output System: boots before the OS

5.3. Network Management

For network managment, it was decided on use of the Network Manager Command Line Interface (nmcli) tool, which is easy to use and offers a sizable set of configuration options and utilities. At the start of the development of the shell scripts for WiFi activation, it was unclear, how to configure a connection in line with the specification of the different WiFi standards. After research on the different security mechanisms and the nmcli documentation was conducted, the solution appeared to be the manual configuration of the encryption algorithm, key management and information element for each connection.

The script developed for changing WiFi passwords was designed to utilize a password list, from which a random password would be extracted for the connection. However, large password lists, such as the RockYou list included in Kali Linux, proved impractical due to their considerable size complicating handling. A password list containing 100,000 entries was selected as the data source. Given the varying requirements for passwords across different WiFi standards, as detailed in chapter 4, the list contained a number of unsuitable entries. It was subsequently filtered using a simple script (see listing 5.1) to generate password lists that conformed to the specific requirements of each standard.

Listing 5.1: script for password extraction

```
#!/usr/bin/bash

lines=$(wc -l < 100kPasswords.txt)
i=1
while [ $i -le $lines ]; do
    currPassword=$(sed -n "${i}p" < 100kPasswords.txt)
    currCharCount=${$currPassword}
    if [ $currCharCount -ge 8 ] && [ $currCharCount -le 63 ]; then
        echo "$currPassword" >> passwordsWPA.txt
    fi
    if [ $currCharCount -eq 5 ] || [ $currCharCount -eq 13 ]; then
        echo "$currPassword" >> passwordsWEP.txt
    fi
    ((i++))
done
```

For the WiFi status function, the **`nmcli dev wifi show-password ifname [device name]`** command is employed to display the type of network, its SSID and password. Unfortunately this command is not functional for WEP networks, therefore a script was created, which extracts the SSID and password from */etc/NetworkManager/system-connections/[connection].nmconnection*, formatting to output alike the nmcli utility.

In the implementation of the WiFi monitor function, the objective was to create a logger that would enable users to trace the events of an attack, such as the dis-/connection of devices and any changes to the network. The command **`iw event -T`** provides timestamps and MAC addresses for devices connecting to or disconnecting from the network. Additionally,

nmcli monitor reports any modifications made to the network status, but does not include timestamps in its output. Attempts to add timestamps by using **awk** or **ts** were unsuccessful, as nmcli monitor must be executed in the background to allow the UI to remain operational. Furthermore, a challenge that remains unresolved is the integration of the outputs from both programs into a single logfile.

As described in chapter 6, there were some critical configuration errors discovered along the testing process. The faulty configuration resulted in the network, supposedly encrypted with the WEP encryption standard, showing up as a WPA network in **airodump-ng**. Also the WPA and WPA2 networks were identified by airodump as WPA3 networks, using SAE as the authentication mechanism. In the pursuit of fixing the configuration errors, the WPA network was accidentally configured to use enterprise authentication in the form on 801.2X. Switching the configuration back to WPA-PSK, the WiFi connection continued trying to reach the RADUIS server for enterprise authentication during activation, timing out as a result.

5.4. Rotary Encoder

To use the encoder dial as a scrollwheel, its signals need to be processed and translated to instructions for the main program to execute. After the encoder was soldered to a simple circuit of pull-up resistors and connected to the GPIO pins of the RPI, the encoder program was developed for processing of the encoders outputs.

Off the shelf rotary encoders incorperate clicks into their rotation, similar to the scrolling wheel of a computer mouse, and output two digital signals. These signals are called DT and CLK, which when turned, oscillate with 90 degrees of phase shift, resulting in a grey code with two bits. Through saveing the previous state and comparing it to the new state, once the encoder was turned, the direction of rotation can be determined. According to various online source, these kind of encoders switch one state on each click like the dotted arrow in fig. 5.2 depicts.

During development of the encoder program, the ouput showed unexpected behavior, which was discovered to originate from the encoder switching through states 0 to 3 between clicks, as depicted by the red arrow in fig. 5.2. The discovery of this behavoir helped to simplify the the processing even easier, since this pattern allows a single query of the signal states to determine the direction of rotation. As seen in listing 4.3, the processing was then applied in a single ISR, comparing the DT and CLK signals.

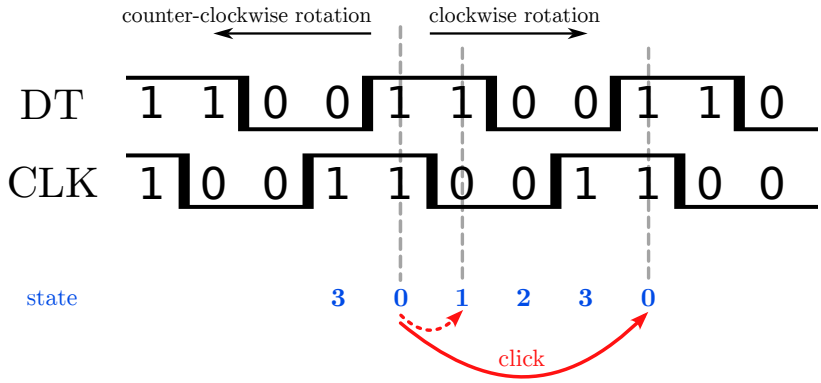


Figure 5.2.: Encoder Processing

5.5. ESP32

To establish communication between the RPI and the ESP32, some of the GPIO Pins needed to be initialized for UART, which again was achieved by adding a hardware overlay to the *config.txt* file. The code of the ESP32 in large part consists of code examples provided by the ESP-IDF github ([Syst 23]). From these examples the code was compiled from snippets the WiFi station example, mqtt websocket example and UART events example. These snippets were modified to work in conjunction with the scripts present on the RPI.

When closing a network and reopening another, instructing the ESP32 to switch to the new network, lead to it regurlarly crash with the error **`itwt_stop_process`**. After investigation, the error seemed to originate from a WiFi power saving mechanism present in WiFi 6, called Target Wake Time (TWT). Various attempts of solving the error by disableing the power saving were attempted, but resulted unsuccessful.

The solution, finally implemented, involed the code beeing restuctured to reset the MCU on a UART command, preceeding the establishment of a network connection.

Chapter 6.

Pentesting

This chapter describes the pentesting of the WiFi enviroments, established with the RCSL. It starts of by explaining some common (WiFi-) network attacks, of which the cracking of WEP and WPA/WPA2 are then tested on the RCSL and the results documented.

6.1. Attacks

WiFi attacks can take many different forms, depending on the type of attack and and which mechanism it tries to break. There are attacks aimed at breaking the Access Controls, Integrity Controls, Confidentiality or Availability. [Oriy 17] The following selection of attacks quite nicely depicts the different vectors an attacker can take to hack a WiFi network and its STAs.

6.1.1. War Driving

War Driving describes the act of gathering information on WiFi networks in larger numbers in order to locate specific or weak targets. This is done by traversing areas of interest with data collection equipment, which collects information about every network coming into range. After capturing and mapping as many WiFi networks as possible, a more sophisticated attack can be launched on the captured networks or the data can be sold on the black market. [Oriy 17]

6.1.2. Rogue Access Point and Evil Twin

When using Rogue Access Points, attackers place extra APs onto a network, trying to stay undetected and get STAs to connect to the malicious AP. This can for example be done by connecting an AP to an open and unprotected LAN port inside a public or company building. Once a STA has connected to the rogue AP it acts as a MITM, allowing the attacker to manipulate or eavesdrop on the traffic.

Evil Twins have some similarity to Rogue Access Points, in that they also pretend to be a legitimate part of an existing network. Different to Rogue APs, Evil Twins do not connect to the legitimate network, but instead spoof wireless networks, providing a stronger signal, to again get STAs to connect to it. Some attackers might also use a deauthentication flood to get STA disconnected from their genuine AP and autoconnect to the malicious one.

Because they employ one-way authentication, WEP and OWE networks are vulnerable to these attacks. [Sank 21, page 105,119]

6.1.3. Cracking WEP

As touched on in chapter 3, WEP has serveral vulnerabilities, most notable the encryption algorithm, the one sided authentication mechanism and the lackluster MIC.

WEP encryption is based on the RC4 algorithm and works "by exclusive-ORing the data stream with a pseudorandom stream of bits generated based on the WEP key" [Sank 21, page 104] and a 24-bit initialization vector (IV). With 24 bits, there are about 16.5 million combinations for the IV, which can be exhausted in a few hours of network traffic. The probability of the same key beeing reused is at about 50% after around 5000 frames have been captured, opening the possibility for reverse engineering by analysis of the frames [Oriy 17].

The MIC with CRC-32 is not sufficient in ensuring message integrity, allowing manipulated frames, injected into the network, to be accepted as non-compromised. [Oriy 17]

Cracking a WEP network involves the reverse engineering of the password, aided by exploiting the authentication mechanism and the MIC. The process begins by scouting the target network, commonly conducted by capturing all traffic on all channels from networks in the vicinity. In normal operation a NIC only recieves the traffic, adressed to its MAC address. To disable this filtering, the network card is put into monitor mode, allowing it to capture all packets transmitted in the vacinity, and discover all APs present. After the target network has been determined, its traffic is captured and after enough frames have been transmitted, the password can be reverse engineered.

However the process can be accelerated by injecting spoofed ARP frames, to artificially increase traffic. This technique is called a replay attack and involves the capture of serveral authentic ARP frames, subsequently modifying and injecting them, which "the AP will re-broadcast [...] and as a result generate [new IVs]" [Oriy 17].

Sending ARP frames without association to the AP will lead to them beeing rejected, therefore another spoofed frame is injected, to fake authentication and association with the AP.

6.1.4. Cracking WPA/WPA2

WPA and WPA2 networks come with improved encryption in the form of TKIP and SAE-CCMP, which cannot be cracked in the way, WEP can [Oriy 17]. To crack a WPA network, it is common to attack the authentication mechanism, because, as described in chapter 3, it can be vulnerable to offline attacks.

An attacker starts off by capturing the handshake between an AP and STA during authentication, from which the PSK is extracted. In order to capture a handshake, the attacker would have to wait for a device to authenticate with the network, which depending on the number of STAs, could take quite some time. Thus a Deauthentication Attack is performed, injecting deauthentication frames, which force the STAs reconnect to the AP and therefore making the capture of a handshake predictable.

Since the PSK is derived from the password and other information, which is openly available, the password can be bruteforced by generating PSKs and comparing them to the captured one. The brute force attack can be accelerated by performing a dictionary attack. However if the network uses a strong password, which is not contained in the password dictionary, the attack will be unsuccessful in the way that it takes too long to brute force the passphrase. Furthermore APs with PMF enabled can not be attacked with this method, since encrypted deauthentication frames cannot be spoofed.

6.1.5. Man-in-the-Middle

6.2. Methods

The Pentests were conducted using the Aircrack-ng suite with Kali Linux. Kali Linux is a Linux distribution, which comes with a vast collection of tools for Pentesting, security auditing and computer forensics. The Aircrack-ng suite is taylored towards network security assessment and is included with Kali Linux.

Note: For more information consult the Kali and Aircrack-ng documentation [Airc 23][Kali 23].

6.2.1. Cracking WEP

Starting the target scouting, **`airmon-ng start wlan1`** is used, which puts the card into monitor mode and creates a new network interface, called "wlan1mon". With the card beeing able to receive all traffic, the target network can be searched, by using **`airodump-ng wlan1mon`**.

```
root@kali: ~
File  Actions  Edit  View  Help

 CH  1 ][ Elapsed: 6 s ][ 2025-04-17 16:41

 BSSID              PWR  Beacons    #Data, #/s  CH   MB   ENC CIPHER  AUTH ESSID

 E4:18:6B:FC:CC:4A  -75        3        0    0   6  130   WPA2 CCMP   PSK  Zyxel_CC4A
 9C:A2:F4:AB:5D:5F  -76        1        0    0   6  130   WPA2 CCMP   PSK  FRITZ!Box 7362 SL_EXT
 4C:FB:FE:6C:CD:36  -75        5        0    0  10  130   WPA2 CCMP   PSK  Vodafone-881207
 56:38:06:2F:B7:EE  -70       10        0    0  11  130   WPA2 CCMP   PSK  AndroidAPa488
 6C:FF:CE:F6:B5:DE  -68       15        7    0  11  260   WPA3 CCMP   SAE  MagentaWLAN-1L2X2
 6C:5A:B0:2C:B9:03  -51       12        0    0   4  130   WPA2 CCMP   PSK  TP-Link_B903
 6C:5A:B0:A6:67:2B  -47       13        0    0  10  270   WPA2 CCMP   PSK  TP-Link_672B
 DE:A6:33:8B:43:97  -60       11        0    0   1  260   WPA3 CCMP   SAE  WLAN
 DC:A6:33:8B:43:94  -60       10        0    0   1  260   WPA3 CCMP   SAE  WLAN von Jonas
 D8:3A:DD:4E:DA:29  -27       17       12    0   1   65   WPA2 CCMP   CMAC RPI
 04:A2:22:4A:75:AC  -74        0        1    0   1  720   WPA2 CCMP   PSK  Familie P+G+M+F
 90:DE:80:95:D6:22  -33       12        0    0   1  130   WEP  WEP         WEPnetwork
 DE:A6:33:8B:43:98  -60       12        0    0   1  260   OPN              Vodafone Homespot
 DE:A6:33:8B:43:96  -60       12        0    0   1  260   OPN              Vodafone Hotspot

 BSSID              STATION            PWR    Rate    Lost   Frames  Notes  Probes

 (not associated)   38:A5:C9:A8:95:75  -68    0 - 1      0        2         MagentaWLAN-1L2X2
 (not associated)   44:17:93:A8:33:D4  -64    0 - 1      0        1         Vodafone-68A9
 DC:A6:33:8B:43:94  AC:72:89:E5:AE:C0  -17    0 - 6e     0        7
 D8:3A:DD:4E:DA:29  60:A5:E2:73:8A:E3  -14   24e- 6e     0       14
```

Figure 6.1.: Screenshot of airodump-ng

As seen in fig. 6.1, the program scans on all channel for networks within reach and lists them in the table on top. The target network, marked with the red border, is listed with its respective BSSID, RSSI, recorded Beacon Frames, encryption standard and cipher.

In the second table, all STAs in reach are listed and to which AP they are associated. If a STA is actively scanning for networks, it is also listed with the network name it is probing for.

After the target network has been identified, airodump-ng is set up with filters to capture all traffic related to the AP and write it into an output file as seen in fig. 6.2.

```
root@kali: ~
File  Actions  Edit  View  Help
┌──(root㉿kali)-[~]
└─# airodump-ng --channel 1 --bssid 90:DE:80:95:D6:22 --write crackWEP wlan1mon
16:53:29  Created capture file "crackWEP-01.cap".

 CH  1 ][ Elapsed: 4 mins ][ 2025-04-17 16:57

 BSSID              PWR RXQ  Beacons    #Data, #/s  CH   MB   ENC CIPHER  AUTH ESSID

 90:DE:80:95:D6:22  -30 100     1937       41    0   1  130   WEP  WEP    OPN  WEPnetwork

 BSSID              STATION            PWR    Rate    Lost   Frames  Notes  Probes

 90:DE:80:95:D6:22  40:4C:CA:5C:F9:CC  -28   54e- 6      0     4619         WEPnetwork
```

Figure 6.2.: Screenshot of airodump-ng filtered for the target network

`--channel` filters the captured frames to only collect frames on the APs channel
`--bssid` filters the frames related to the BSSID of the target
`--write` specifies the output file
All devices associated with the network can be seen in the lower table, which at that time is only the ESP32 with the address **`40:4C:CA:5C:F9:CC`**.

For the replay attack, a fake association with the AP is established, using **`aireplay-ng`** as seen in fig. 6.3.
`--fakeauth 0` defines the authentication to be performed infinitely, until a connection is established.
`-e` sets the name of the target network
`-a` sets the BSSID of the target network
`-h` sets the MAC of the device to be associated with the target network

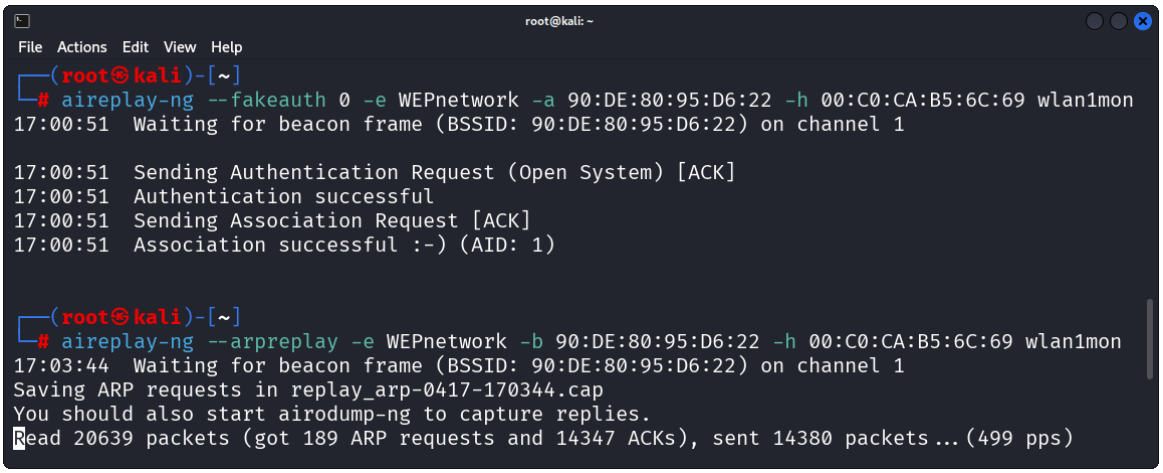


Figure 6.3.: Screenshot of aireplay-ng fake authentication and ARP replay attack

Upon successful association, to be seen in fig. 6.3, ARP requests can be injected.
`--arpreplay` selects the replay attack for ARP frames
`-b` sets the BSSID of the target network
`-h` sets the MAC address of the attacker

Creating and capturing traffic, IVs are collected and the cracking of the password can be started simlutaniously. As seen in fig. 6.4, **`aircrack-ng`** is executed and reads the output of airodump-ng, attempting to crack the password for every 5000 IVs collected.

```
root@kali: ~
File  Actions  Edit  View  Help
┌──(root㉿kali)-[~]
└─# aircrack-ng crackWEP-01.cap
Reading packets, please wait...
Opening crackWEP-01.cap
Read 157292 packets.

   #  BSSID              ESSID                     Encryption

   1  90:DE:80:95:D6:22  WEPnetwork                WEP (1680 IVs)

Choosing first network as target.

Reading packets, please wait...
Opening crackWEP-01.cap
Read 157292 packets.

1 potential targets

Attack will be restarted every 5000 captured ivs.

                                                Aircrack-ng 1.7


                                [00:00:04] Tested 167297 keys (got 1707 IVs)
                                           Got 2196 out of 5000 IVs
   KB    depth   byte(vote)
    0   59/ 60   FD(2304) 01(2048) 08(2048) 0D(2048) 10(2048) 12(2048) 19(2048) 29(2048)
    1   13/  1   FC(3072) 1A(2816) 28(2816) 36(2816) 5A(2816) 5C(2816) 6C(2816) 83(2816)
    2    7/ 22   C2(3328) 39(3072) 4C(3072) 5B(3072) 74(3072) C0(3072) EE(3072) F6(3072)
    3    1/  4   D6(3840) BD(3584) 16(3328) 8C(3328) 06(3072) 1F(3072) 2A(3072) 38(3072)
    4   21/  4   F2(2816) 03(2560) 06(2560) 0C(2560) 13(2560) 1E(2560) 25(2560) 52(2560)

Failed. Next try with 5000 IVs.
```

Figure 6.4.: Screenshot of the cracking process with aircrack-ng

6.2.2. Cracking WPA/WPA2

This attack starts off by scouting the target network with **`airodump-ng`** as well, in this case it is called "WPAnetwork".

Once the desired AP has been discovered, similar to Cracking WEP, **`airodump-ng`** is used to capture the (re-)authentication handshake. The Deauthentication Attack is performed with **`aireplay-ng`** to force the re-authentication of connected STAs (see fig. 6.5).

`--deauth 10` promts the program to transmit 10 deauthentication frames
`-a` specifies the APs BSSID, leading to the frames beeing broadcasted to all STAs
`-c` can be used to specify the MAC address of a STA, to send the frames directly to one STA, as recommended by the program

After capturing the authentication handshake, a dictionary attack is performed to crack the password, which in this case utilizes the "rockyou.txt" password list, built into kali linux, to brute force the password.

```
root@kali: ~
File  Actions  Edit  View  Help
┌──(root㉿kali)-[~]
└─# aireplay-ng --deauth 10 -a 90:DE:80:95:D6:22 wlan1mon
17:19:31  Waiting for beacon frame (BSSID: 90:DE:80:95:D6:22) on channel 13
NB: this attack is more effective when targeting
a connected wireless client (-c <client's mac>).
17:19:31  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
17:19:32  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
17:19:32  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
17:19:33  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
17:19:33  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
17:19:34  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
17:19:34  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
17:19:35  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
17:19:35  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
17:19:35  Sending DeAuth (code 7) to broadcast -- BSSID: [90:DE:80:95:D6:22]
```

Figure 6.5.: Screenshot of the Deauthentication Attack

`aircrack-ng -b [bssid of router] -w [path to word list] [path to capture packets]` starts to interate through the password list, computes the PSK for each password and compares it to the captured PSK.

6.3. Results

6.3.1. Cracking WEP

The first attempt at cracking the WEP network was unsuccessful, because only the ESP32 beeing connected to the network, ARP request injection cannot generate significant traffic, therefore the collection of IVs proceeds very slowly. After X time and only Y captured IVs, the test was cancelled.

As a workaround for test two, the RPI was connected to the internet and a second STA in form of a Smartphone was put on the network. The smartphone streamed a video, which generated sufficient traffic. This time the password could be cracked after around 3 minutes and only 214 captured IVs.

For control, a third test was conducted, for which the password of the hotspot was changed with the new password function. Again the password could be cracked in little time, with around 15000 IVs collected.

An investigation into the first test revealed, that the ARP injection did not function properly, therefore a fourth test without the video stream was conducted. With working ARP injection,

```
root@kali: ~
File  Actions  Edit  View  Help

                                                    Aircrack-ng 1.7

                                        [00:06:24] Tested 44 keys (got 14987 IVs)
                                      Got 15009 out of 15000 IVsStarting PTW attack with 15009 ivs.
  KB    depth   byte(vote)
   0    1/  3   74(20736) CB(20224) 9F(19200) BD(19200) EA(18944) F2(18944) 2C(18688) 15(18176) 49(18176) 4D(18176)
   1    1/  3   EC(20480) D9(19968) D6(19456) 2B(19200) 6D(19200) 7D(19200) 93(19200) D1(19200) 96(18944) 1B(18688)
   2    0/  1   73(24320) 30(19968) 74(19968) 68(19712) 19(19456) 26(19200) 3F(19200) EA(19200) 55(18688) A9(18688)
   3    0/  3   74(20480) E4(20224) BA(19712) BE(19200) 83(18944) 89(18944) 8E(18944) 31(18688) 3A(18688) 8C(18688)
   4    0/  2   31(22272) 5A(22016) 17(19456) 5F(19456) 9C(19456) 05(18944) A0(18944) AE(18944) EF(18944) 62(18688)

                    KEY FOUND! [ 74:65:73:74:31 ] (ASCII: test1 )
      Decrypted correctly: 100%
```

Figure 6.6.: Screenshot of the fourth cracking attempt

the IV collection proceeded quicker than in test one, however it still took around 20 minutes of frame capture and six minutes of cracking, as depicted in fig. 6.6, to find the password.

6.3.2. Cracking WPA/WPA2

Upon the first test it was discovered, that the configurations for the WPA and WPA2 networks were not correctly set up (see chapter 5).

```
root@kali: ~
File  Actions  Edit  View  Help

┌──(root㉿kali)-[~]
└─# aircrack-ng -b 90:DE:80:95:D6:22 -w /usr/share/wordlists/rockyou.txt crackWPA-01.cap
Reading packets, please wait...
Opening crackWPA-01.cap
Read 1304 packets.

1 potential targets

Packets contained no EAPOL data; unable to process this AP.


Quitting aircrack-ng...
```

Figure 6.7.: Screenshot of the failed handshake capture

Chapter 7.

Discussion

In this chapter, the importance of education in cybersecurity will be highlighted and the potential use cases for the RCSL in education will be discussed.

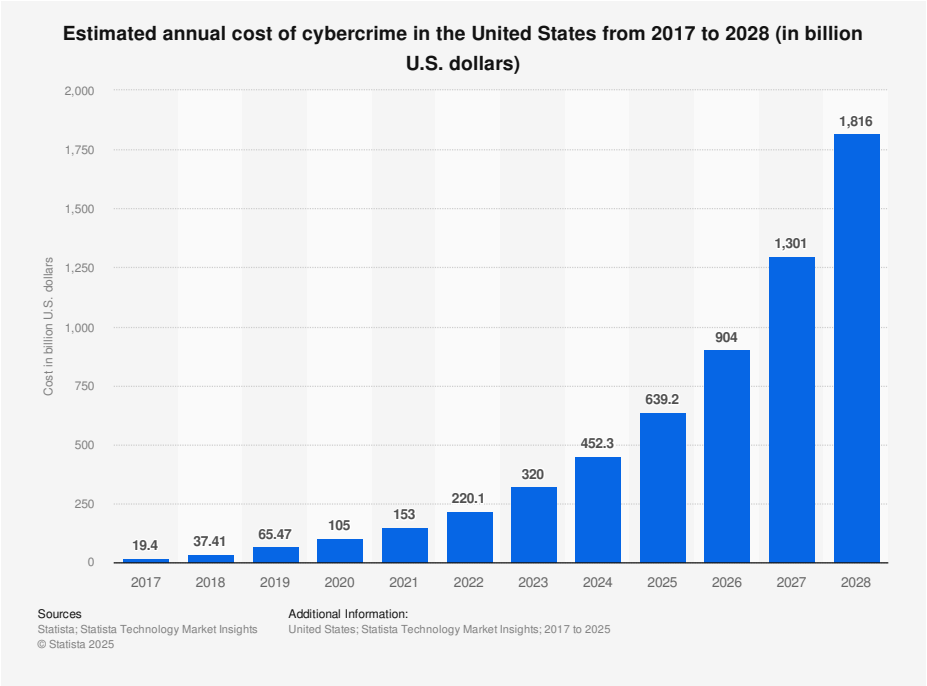
7.1. Importance of Cybersecurity Education

The advancement of digital information technology, accelerated by trends like Artificial Intelligence, Cloud Computing or the Internet of Things (IoT), has reached a high pace, which companies are trying to keep up with. In the pursuit of keeping up with this pace, convenience is often the main focus, whereas security and privacy are left as an afterthought. [Salm 17, chapter 1] The consequences of an attack can be unavailability of services and loss of valuable or sensitive data. Expert estimate, that cybercrime causes annual damages of 110 billion dollars worldwide and most companies, relying heavily on their IT infrastructure, can only operate for a few days without these systems [Hell 23, page 3],[Salm 17, page 12].

Cyberattacks in the form of data piracy, or blackmailing have become common place and cause ever increasing financial damages for companies, the public and governments, as seen on the example of US companies in section 7.1.

Cybersecurity awareness and education plays a large role in ensuring the integrity of networks and systems, since a system is only as strong as its weakest link. People without awareness and training in cybersecurity and will oftentimes tend to circumvent or ignore security mechanisms, since following the principles of cyber security is often connected to effort. [Hell 23] Awareness also helps to prevent people from falling for fishing schemes like fraudulent emails or usb stick baiting.

Therefore raising among the members of an organization helps to mitigate the risks of a system beeing breached by common attacks such as phishing. Awareness of threads in the Cyber Space acts as the fundation for behaviour in compliance with Cybersecurity principles.



The RCSL is able to create test environments for various applications and could be used as tool in education for demonstration purposes and to allow students to get hands-on experience. As an example the device could be used by a lecturer to demonstrate the easy with which data can be stolen or manipulated, if a WiFi network is not or badly secured. This can help the listeners to become aware of the risk, associated with the use of open networks, and the importance of using secure standards and adhering to certain principles.

7.2. Pentesting Results Evaluation

Concluding the tests, it appears as the Cracking of WEP networks can be performed reliably with the RCSL. However the generation of artifial network traffic could aid the speed of the process, as well as depict a more realistic scenario, and might be considered for future improvements.

7.3. Usecase

Chapter 8.

Summary

Chapter 9.

Outlook

9.1. UI Program

The UI Program is using the navigate() function, explained in chapter 4 to periodically reprint the display and read the user input. To improve performance, the function could be made to only refresh the output on change, meaning an interrupt on user input could trigger the reprinting of the menu and selection.

A menu manager class could be added, which contains a menu stack and methods to manipulate it. Instead of implementing navigate() as a method of the menu class, it could be a method of the menu manager, to display the menu currently on top of the menu stack.

The structure and therefore readability of the code could furthermore be improved by replacing the lambda functions and implementing the functions as methods with the decorator pattern.

As described in chapter 5, the setup without a graphic server does limit the scaling of the UI to adjusting the resolution in the display configuration in config.txt. The UI could be reworked with a graphical interface to allow individual scaling of the menu items and the terminal output to improve the outputs readability.

9.2. Input Processing

The encoder program could be merged into the UI program and run as a seperate thread, which sends the preprocessed inputs to the parent thread. This change would allow the parallel input via the rotary encoder and a keyboard, which could be useful, when a developer is connected to the RPI over SSH.

9.3. Features

There are serveral features, which can be implemented in the future to greatly improve the devices versatility. Adding battery power would improve the portability of the device, but would most likely also require a more efficient SBC than the RPI. In the field of wireless communication, Bluetooth, NFC and 2.4Ghz RF communication environments could be set up for pentesting. Pentesting with SSL/TLS attacks and Malware injection could be implemented as well as the addition of dummy files to test the cracking of different cryptographic techniques. The addition of defensive features like an Intrusion Detection and Prevention System would be a great way of teaching the application of cyber security techniques.

Appendix A.

Supplemental Information

A.1. User Manual

Anleitung zum Raspi-CyberSec-Lab

Jonas Schmitt

5. April 2025

1 Allgemeine Informationen

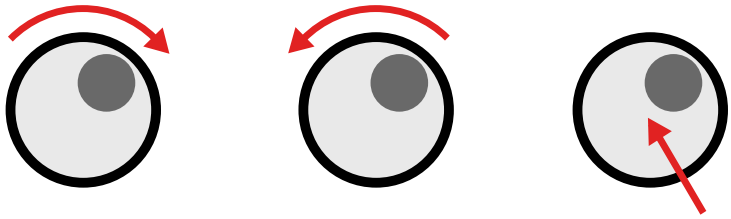
Das RaspberryPI Cybersecurity Lab ist eine Platform zum Testen verschiedener Cyberangriffe, das Projekt ist Teil meiner Bachelorarbeit in Elektro- und Informationstechnik an der Technischen Hochschule Nürnberg. Es hat verschiedene Funktionen um Umgebungen zu schaffen, welche für das Ausprobieren von Angriffen aus den Bereichen Wifi Netzwerke, Webseiten / -applikationen, MQTT Kommunikation und Bluetooth ausgelegt sind.

2 Benutzeranleitung

Zum Einschalten des Geräts, schließen Sie das Gerät mit dem zugehörigen Netzteil an den Strom an. Das Gerät fährt nun hoch, wobei der Bootvorgang zu sehen ist (das RaspberryPI Logo wird angezeigt). Nach erfolgreichem Bootvorgang wird das Hauptmenü angezeigt.

2.1 Navigation

Zur Navigation wird das Drehrad, wie in folgender Abbildung beschrieben, genutzt:



nach unten **nach oben** **bestätigen**

Die farbige Hinterlegung zeigt an, welcher Menüpunkt aktuell ausgewählt ist. Mit dem Drücken des Knopfes wird die Auswahl bestätigt und das Untermenü geöffnet, bzw die Aktion ausgeführt.
Zum verlassen eines Menüs wird die Option "back"gewählt.

2.2 Menü Übersicht

Dem Nutzer steht der folgende Menübaum zu Verfügung:

- **wifi** - Optionen zu Wifi Netzwerken
  - **activate** - Aktivieren eines Hotspots
    Auswahl der Netzwerkart
  - **deactivate** - Abschalten des aktiven Hotspots
  - **status** - Anzeigen der aktuellen Netzwerkeinstellungen
  - **change password** - Setzen eines neuen Passworts
    Auswahl der zu änderenden Netzwerkart
  - **monitoring** - Einschalten und Anzeigen des Netzwerkmonitoring
    on - Einschalten
    off - Ausschalten
    show log - Log anzeigen
    delete log - Log löschen
- **bluetooth** - Optionen zu Bluetooth
- **webapp** - Optionen zu Webapplikationen
  - **Juice Shop**
    on - Server Einschalten
    off - Server Ausschalten
  - **MQTT**
    on - MQTT Konversation starten
    off - MQTT Konversation beenden
- **development** - Anpassbare Optionen für Entwickler
- **power off** - Ausschalten des Geräts

3 Installation

Das Projekt wurde mit folgender Hardware umgesetzt:

- RaspberryPI 4B mit 8GB RAM
- Fenvi AX1800 USB Netzwerkkarte
- Waveshare ESP32c6 Microcontroller
- Waveshare 4.3 zoll LCD
- Drehgeber mit Druckknopf

Als Betriebssystem des RaspberryPI dient RaspberryPI OS in der light Version (nur Kommandozeile). Zum Kompilieren und Flashen des ESP32 Codes wird das ESP-IDF benötigt, dieses kann in Visual Studio Code als Erweiterung installiert werden.

Zuerst muss das Github Repository des Projekts auf den RaspberryPI geklont werden, dies kann mit folgendem Kommando erfolgen:

```
git clone https://github.com/Der-Erzfeind/Raspi-CyberSec-Lab-Project.git
```

Es wird empfohlen das Repository in das standardmäßige Nutzerverzeichnis zu klonen. Folgende Aktionen müssen im Anschluss ausgeführt werden:

- Kompilieren von main.cpp, encoder.cpp und mqtt.cpp
- Alle Skripte ausführbar machen
- Installation und Konfiguration von Mosquitto
- Installation des JuiceShops (und node.js)
- Einrichten des systemd services zum Ausführen von start.sh beim Start des Geräts
- Kompilieren und Flashen des ESP32 Codes

A.2. WiFi Basics

Table A.1.: Evolution of Wi-Fi Security [Sank 21, page 123]

	WEP	WPA	WPA2	WPA3
Certification start	-	2003	2004	2018
Mandatory since	-	2006	2006	2019
Underlying standard(s)	802.11	802.11i(part. compliance)	802.11i, 802.11w(PMF)	802.11s(SAE), 802.11w
Information element	None	WPA IE	RSN IE	RSN IE
Encryption	RC4	TKIP	TKIP, AES-CCMP	AES-CCMP, AES-GCMP
Authentication	None	PSK, 802.1X	PSK, 802.1X	SAE, 802.1X
Usability enhancement	None	None	WPS	DPP
Open Wi-Fi security	None	None	None	OWE

A.3. Documentation

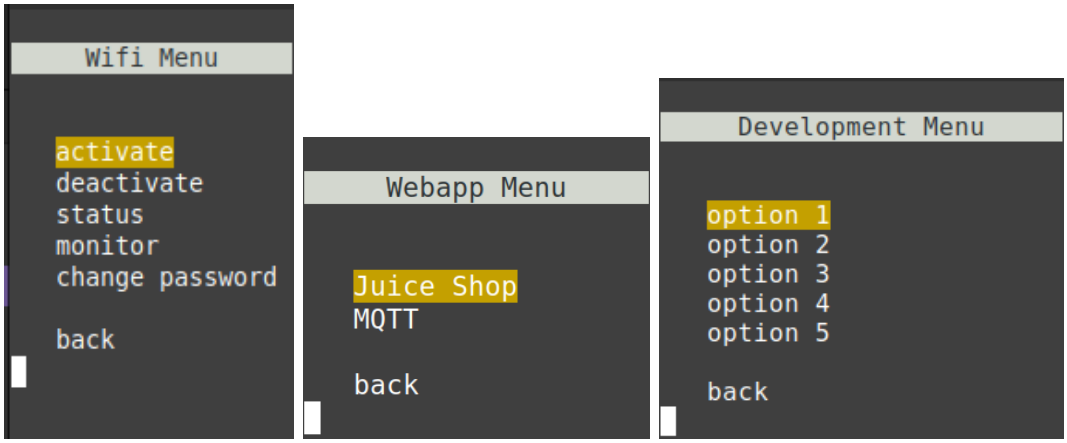


Figure A.1.: Device Menues

Table A.2.: menu and option overview

Main Menu						
wifi		bluetooth	webapp		development	power off
activate	WEP	*under*	Juice Shop	on	option 1	
	WPA	*development*		off	option 2	
	WPA2		MQTT	on	option 3	
	WPA3			off	option 4	
deactivate					option 5	
status						
new password	WEP					
	WPA					
	WPA2					
	WPA3					
monitor	on					
	off					
	show log					
	delete log					

List of Figures

3.1. MAC Frame of BSS WiFi Network 9
3.2. Connection Establishment 11

4.1. Hardware Components 15
4.2. Software Architecture 17
4.3. Main Menu 18
4.4. Navigation Scheme 19
4.5. WiFi status menu 20

5.1. Concept Model 23
5.2. Encoder Processing 27

6.1. Screenshot of airodump-ng 32
6.2. Screenshot of airodump-ng filtered for the target network 32
6.3. Screenshot of aireplay-ng fake authentication and ARP replay attack 33
6.4. Screenshot of the cracking process with aircrack-ng 34
6.5. Screenshot of the Deauthentication Attack 35
6.6. Screenshot of the fourth cracking attempt 36
6.7. Screenshot of the failed handshake capture 36

A.1. Device Menues 47

List of Tables

A.1. Evolution of Wi-Fi Security [Sank 21, page 123] 47
A.2. menu and option overview 48

List of Listings

4.1. wifiActivate function in the UI program 18
4.2. use of wifiActivate for WEP network 18
4.3. Rotation handling ISR 19
4.4. extract of wifiActivate.sh 20
4.5. extract from wifiNewPassword.sh 21

5.1. script for password extraction 25

Bibliography

[Airc 23] Aircrack-ng. "Aircrack-ng Documentation". 2023. Accessed: 2023-10-01.

[Cybe 23] *Cybersecurity Teaching in Higher Education.* Springer Cham, 2023.

[Geek 23a] GeeksforGeeks. "IEEE 802.11 mac Frame". Mar 2023.

[Geek 23b] GeeksforGeeks. "Intrusion Detection System (IDS)". 2023. Accessed: 2023-10-01.

[Hark 08] D. Harkins. "Simultaneous Authentication of Equals: A Secure, Password-Based Key Exchange for Mesh Networks". In: *2008 Second International Conference on Sensor Technologies and Applications (sensorcomm 2008)*, pp. 839–844, 2008.

[Hell 23] R. Hellmann. *It-sicherheit: Methoden und Schutzmaßnahmen Für Sichere Cybersysteme.* De Gruyter Oldenbourg, 2023.

[Kali 23] Kali Linux. "Kali Linux Documentation". 2023. Accessed: 2023-10-01.

[Kimm 25] B. Kimminich. *Pwning OWASP Juice Shop.* Leanpub, 2025.

[Mari 24] H. S. Mariano and D. C. Café. "Measuring Public Wi-Fi Security Awareness via Captive Portal Connections Using a Microcontroller". In: *9th Workshop on Communication Networks and Power Systems (WCNPS 2024)*, University of Brasília (UnB), Brasília, Brazil, 2024.

[NR 25] M. NR. "USB WiFi Adapters that are Supported with Linux In-Kernel Drivers". 2025. Accessed: 2025-03-20.

[Oriy 17] S.-P. Oriyano. *Kali Linux Wireless Penetration Testing Cookbook: Identify and assess vulnerabilities present in your wireless network, Wi-Fi, and bluetooth enabled devices to improve your wireless security.* Packt Publishing, 2017.

[Pogu 13] W. Poguntke. *Basiswissen IT-Sicherheit.* W3L-Verlag, 2013.

[Salm 17] A. Salmon, W. Levesque, and M. McLafferty. *Applied Network Security: Master the art of detecting and averting advanced network security attacks and Techniques.* Packt Publishing, 2017.

[Sank 21] S. G. Sankaran and S. R. Gulasekaran. *Wi-Fi 6: Protocol and Network.* Artech House, 2021.

[Syst 23] E. Systems. "ESP-IDF". https://github.com/espressif/esp-idf/tree/master, 2023. GitHub repository.

[Watj 18] D. Wätjen. *Kryptographie - Grundlagen, Algorithmen, Protokolle*. Vol. 3. Auflage, Springer Vieweg, 2018.

[Wiki 25] Wikipedia. "Wi-Fi". Mar 2025.

Glossary

GPIO General Purpose Input/Output are pins on a computing devices, which can be configured to read or write data. i

library A suite of reusable code inside of a programming language for software development. i

shell Terminal of a Linux/Unix system for entering commands. i

UART Universal Asyncronous Receiver/Transmitter is a bus standard for serial communication between embedded devices. i